TU Clausthal
Clausthal University of Technology

Crane control system with integrated anti sway module

Project Overview

Amer El-Saad
Professor: Stefan Palis
Supervisor: Mykhailo Kutia

Date: 24 April 2024

1. Introduction

Load sway during crane operation represents a significant challenge in industrial and construction environments. This phenomenon not only increases the risk of accidents but also substantially lowers operational efficiency and causes additional stresses on crane components, leading to increased wear and maintenance requirements [1]. When loads swing uncontrollably, they create dangerous situations for workers, potentially damage surrounding equipment, and extend operation times as operators must wait for oscillations to dampen naturally before precise positioning can be achieved [2].

Anti-sway control systems have emerged as a critical solution to mitigate these issues by stabilizing the load in real-time, making crane operations safer, more efficient, and more precise. Often, the load position is measured using specialized markers and computer vision systems, which provide real-time feedback to the control mechanism [3].

To guide the development of this project, a set of tasks was outlined and tracked throughout the process. These tasks helped structure the implementation and ensured all core components were addressed.

Task 1: Augment the current crane setup with a camera and add an ArUco marker to the crane hook.

A camera was mounted on the crate of the crane, positioned to face downward toward the load. An ArUco marker was attached to the crane hook for visual tracking.

Task 2: Implement an appropriate computer vision system, e.g., using the ArUco library of the OpenCV project.

The computer vision system was implemented in Python using the OpenCV ArUco library. The camera stream is processed to detect the marker in real-time.

Task 3: Implement a first anti-sway control algorithm using the sway angle as the controlled variable and the drive velocity as the control handle.

The anti-sway control algorithm was developed and programmed in Arduino. The sway angle (θ) calculated from the marker position is used to adjust the crane's drive velocity to counteract swinging.

Task 4: Analysis and Evaluation:

We have plotted graphs that compare system behavior with and without the anti-sway control. These visualizations clearly show the reduced oscillation when the control is active.

In recent years, the use of low-cost hardware and open-source libraries has made it easier to develop experimental control systems for educational and research purposes. Technologies like Arduino, OpenCV, and ArUco markers offer powerful and affordable tools for detecting load positions and implementing real-time control responses [4][5][6]. This allows researchers and students to explore advanced control strategies in smaller setups without needing expensive industrial equipment.

This project takes advantage of these accessible technologies to build an anti-sway control system using a scaled-down crane model. The system combines computer vision (using ArUco markers and a USB camera), real-time motor control (through an Arduino Uno), and a simple feedback-based control algorithm to reduce load oscillations. This setup allows for the testing of real-time control logic in a controlled, safe environment and demonstrates how theoretical concepts like sway angle feedback and proportional velocity adjustment can be applied in practice.

While anti-sway technology has been implemented in large-scale industrial cranes, there remains significant value in developing and testing these systems on smaller scale models [7]. This research project focuses on implementing an anti-sway control system for a scaled-down crane model, which serves as an excellent platform for experimentation and demonstration of control principles without the risks and costs associated with full-scale implementations.

The primary challenge addressed in this project is the development of an effective anti-sway control system for a scaled-down crane model. The system must be capable of detecting load position in real-time and implementing appropriate control strategies to minimize oscillations. This requires integration of computer vision capabilities, precise motor control, and implementation of control algorithms suitable for the physical characteristics of the crane system.

The specific problems that need to be solved include:

- Accurate detection and tracking of the load position using computer vision
- Real-time communication between the vision system and the motor controller
- Implementation of control algorithms that can effectively dampen oscillations
- Integration of hardware components into a cohesive and functional system
- Optimization of system parameters to achieve stable performance

The aim of this project is to develop a comprehensive anti-sway control system for a slewing Scaled-down crane model that demonstrates the principles and effectiveness of computer vision-based control. The following objectives have been established:

- Augment the current crane setup with a camera system and add an ArUco marker to the crane hook for position tracking
- Implement an appropriate computer vision system using the ArUco library of the OpenCV project
- Develop and implement an anti-sway control algorithm using the sway angle as the controlled variable and the drive velocity as the control handle
- Analyze the performance of the resulting control system under various conditions
- Document the entire development process, implementation details, and performance results

2. Project Setup

2.1 Scope and Limitations

This project focuses specifically on implementing anti-sway control for a scaled-down crane model using Arduino-based control and computer vision. The scope includes:

- Hardware integration of Arduino Uno, H-bridges, motors, and camera system
- Software development for both Arduino (motor control) and Python (computer vision)
- Implementation of basic anti-sway algorithms with potential for future enhancements
- Performance testing under controlled conditions
- Documentation of the complete system

The project has several limitations that should be acknowledged:

- The system is implemented on a Scaled-down crane model crane, which has different dynamics from industrial-scale cranes
- The 9V DC motors used have limited torque
- The computer vision system is sensitive to lighting conditions and camera positioning
- The control algorithms implemented represent a starting point rather than an optimized solution
- The system currently focuses on single-axis control (trolley movement) rather than multi-axis stabilization
- The presence of numerous black dots on the floor interfered with the camera's ability to reliably detect the ArUco marker, reducing the accuracy of the computer vision system.

2.2 Methodology Overview

The methodology employed in this project combines hardware integration, computer vision, and control theory to create a functional anti-sway system. The approach can be summarized as follows:

- *Hardware Setup:* Integration of Arduino Uno with H-bridges to control the crane motors, with particular focus on the trolley movement motor for anti-sway control. A high-definition camera is mounted to observe the load position.
- *ArUco Marker Implementation:* An ArUco marker is attached to the crane hook, providing a standardized pattern that can be reliably detected by computer vision algorithms.
- *Computer Vision System:* A Python script utilizing OpenCV processes the camera feed to detect the ArUco marker and calculate its position relative to a defined center point.
- *Control Algorithm:* The Arduino receives position data from the Python script and implements a control algorithm that adjusts motor speed and direction to counteract swaying motions.
- *Testing and Refinement:* The system is tested under various conditions to evaluate its performance and identify areas for improvement.

This approach allows for a modular system where individual components can be refined or replaced as needed, providing flexibility for future enhancements such as the implementation of more sophisticated control laws like the sliding mode control.

In the following sections, we will delve deeper into the hardware components, software implementation, and performance results of this anti-sway control system for the Scaled-down crane model.

2.3 Hardware Components

The hardware implementation of the anti-sway control system for scaled-down crane model consists of several integrated components that work together to enable precise control and stabilization. This section details each hardware element, its specifications, and its role within the overall system.

The complete hardware setup comprises the following main components: an Arduino Uno microcontroller (with extension prototype shield), two H-bridge motor drivers, three DC motors for different crane movements, an ELP 1080P Full HD USB camera for ArUco marker detection, and an ArUco marker attached to the crane hook. To minimize oscillation amplitude and increase the natural frequency of the swinging load, the rope length was fixed at 17 cm. This design choice is based on the physical relationship $\omega = \sqrt{\frac{g}{l}}$, where a shorter length l results in a higher angular frequency ω, thereby reducing the duration and severity of the sway. Although the length is currently fixed, the system can be modified in the future to accommodate changes in rope length, making the control strategy adaptable to different physical configurations. A custom 3D-printed enclosure was used to house the electronics, and a computer system runs Python for image processing and control signal generation.

These components are interconnected to form a closed-loop control system where the camera detects the position of the ArUco marker, the computer processes this information, and the Arduino controls the motors to stabilize the load.

Arduino Uno Controller. The Arduino Uno serves as the main controller for the system, processing input signals and generating appropriate output signals to the H-bridges and motors. The Arduino Uno was selected for this project due to its reliability, ease of programming, and sufficient I/O capabilities for the required control tasks (Fig.1)

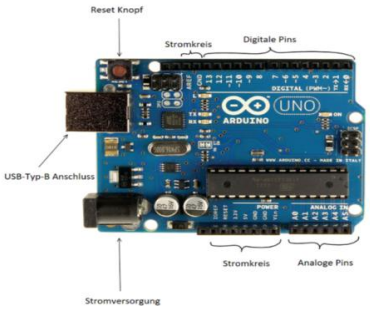

Figure 1: Arduino Uno

The Arduino Uno features: - ATmega328P microcontroller - 14 digital input/output pins (of which 6 can be used as PWM outputs) - 6 analog inputs - 16 MHz crystal oscillator - USB connection for programming and communication - Power jack for external power supply - Operating voltage of 5V.

Extension prototype shield: an extension prototype shield is mounted on top of the Arduino Uno to provide additional GND and +5V connections, simplifying the wiring process and improving the reliability of the connections. This shield also provides a more organized platform for connecting the various components of the system.

*Motor control pins**:*** the Arduino mega microcontroller interfaces with the crane system through the following pin configuration: motor 1 (crane body rotation): PWM speed control: digital pin 3, direction control:in3: digital pin 11, in4: digital pin 12, motor 2 (hook vertical movement): PWM speed control: digital pin 6, direction control:in1: digital pin 8, in2: digital pin 9, motor 3 (trolley movement): PWM speed control: digital pin 10, direction control: in3: digital pin 4, in4: digital pin 5.

Joystick input pins: trolley control (x-axis): analog pin a0, hook control (y-axis): analog pin a1, crane body rotation (z-axis): analog pin a2.

Two H-bridge motor drivers are used to control the three dc motors of the crane. H-bridges allow for bidirectional control of the motors, enabling both forward and reverse movements with variable speed control through PWM signals. the first H-bridge controls two motors: motor 3 (trolley movement) - this motor is the primary focus for the anti-sway control system as it controls the horizontal movement of the trolley. - connections to Arduino: - in3 (yellow wire) → pin 4 - in4 (yellow wire) → pin 5 - PWM (green wire) → pin 10 - connections to crane: - blue and green wires connect to the crane's motor, motor 2 (hook vertical movement) - connections to Arduino: - in1 (orange wire) → pin 8 - in2 (orange wire) → pin 9 - PWM (yellow wire) → pin 6 - connections to crane: - white and red wires connect to the crane's motor. the second h-bridge controls one motor: motor 1 (crane body rotation) - connections to Arduino: - in3 (brown wire) → pin 11 - in4 (brown wire) → pin 12 - PWM (yellow wire) → pin 3 - connections to crane: - orange and yellow wires connect to the crane's motor. A schematic of this configuration is provided in Fig. 2

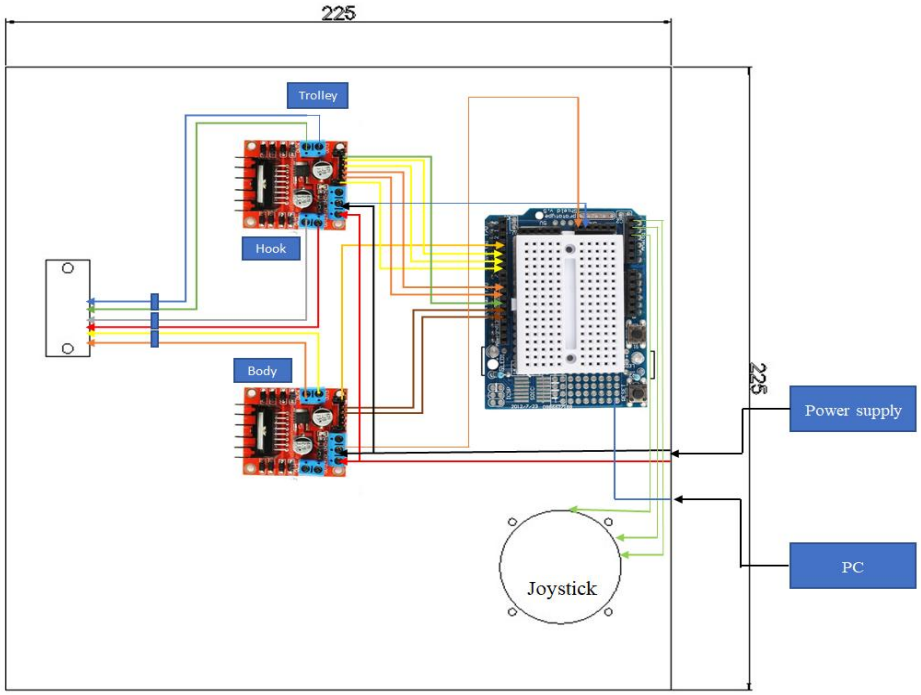

Figure 2: Schematic of Arduino connections

2.4 Camera System

The vision system is a critical component of the anti-sway control mechanism, providing real-time position data of the load.

Camera Specifications: ELP 1080P Full HD OTG USB 2.0 (Fig.3) industrial cameras were used in this project, with the following specifications: - Resolution: 1920 x 1080 pixels (Full HD) - Frame rate: Up to 100 fps - Interface: USB 2.0 - Sensor: High-quality CMOS sensor - Lens: 2.8mm wide-angle lens

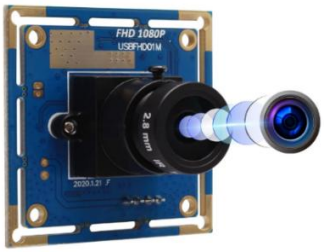

Figure 3: Camera used in Project

Camera Mounting and Positioning: The first camera was mounted on the crane's crate, facing downward to observe the load. This positioning allows the camera to detect the ArUco marker attached to the load and track its position relative to a fixed reference point.

Camera Function in Control System: The primary camera's function is to detect the ArUco marker on the load as showed in the Fig.4 and calculate the deviation between the camera's

center point (represented by a red dot in the visualization) and the center of the ArUco marker (represented by a green dot). This deviation is transmitted as a control signal to the Arduino, which then actuates the crate's motor to align both centers, thereby reducing and preventing continued oscillation of the load.

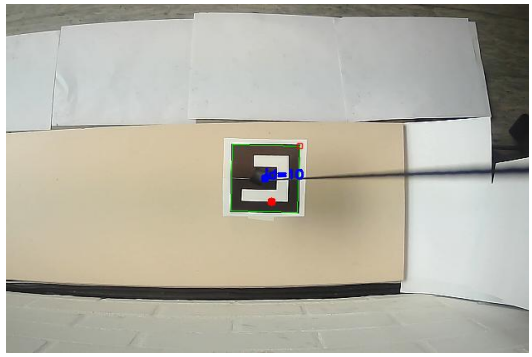

Figure 4: The marker as seen from the camera mounted on the crane

2.5 ArUco Marker Implementation

ArUco markers are synthetic square markers composed of a wide black border and an inner binary matrix that determines its identifier. These markers are widely used in computer vision applications due to their robust detection capabilities even under varying lighting conditions and perspectives.

Marker Selection: for this project, markers from the 4x4 dictionary (DICT_4X4_50) were selected, which offers a good balance between detection reliability and encoding capacity. The specific marker with ID 10 was chosen for the implementation Fig.5.

Marker Placement: The ArUco marker was printed and attached to the load that hangs from the crane hook. The marker was sized appropriately (100mm) to ensure reliable detection from the camera's mounting position. The high contrast black and white pattern of the ArUco marker makes it easily distinguishable from the background, facilitating robust detection.

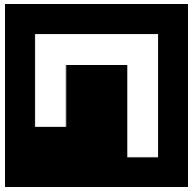

Figure 5: Aruco Marker

2.6 Physical Assembly

The physical assembly of the system involved integrating all electronic components into a cohesive unit and mounting them on the scaled-down crane model structure.

Custom Enclosure: All electronic components, including the Arduino, H-bridges, and wiring, were enclosed in a custom 3D-printed gray box. This enclosure was designed to

protect the electronic components while providing a dedicated space for the joystick to extend for user interaction (Fig.6).

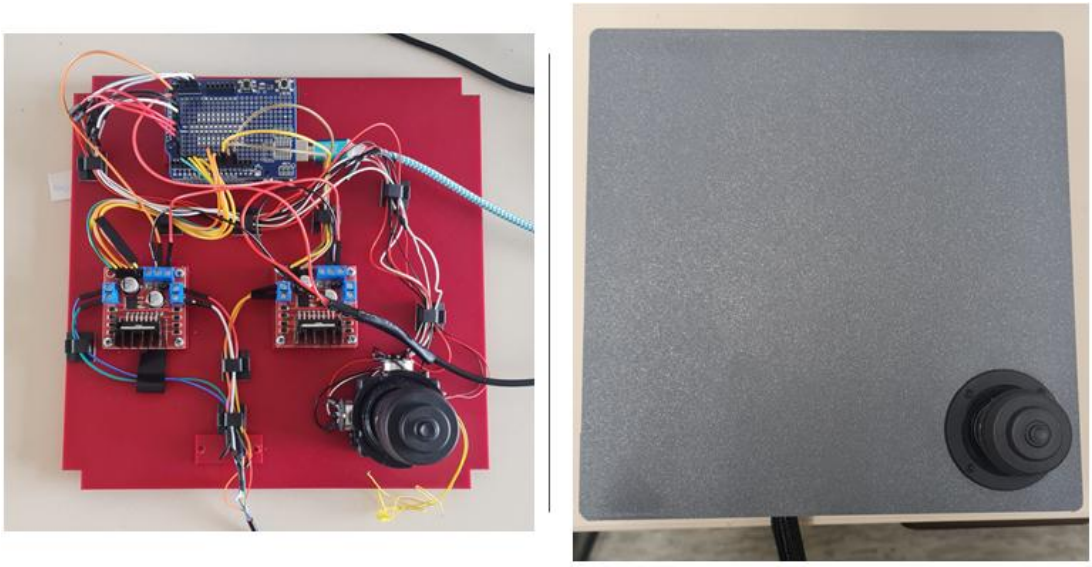

Figure 6: Final enclosure of the system

Wiring and Connections: The wiring between components follows the pin assignments detailed earlier. Color-coded wires were used to maintain organization and facilitate troubleshooting: - Power connections: Red for +5V, Black for GND - Signal wires: Various colors (Yellow, Green, Orange, etc.) for different control signals

Final Integration: The final integrated system includes: - The custom enclosure containing the Arduino and H-bridges - Cameras positioned at strategic locations for optimal marker detection - ArUco markers placed on the load and crate - All necessary wiring between components - Connection to a computer running the Python vision processing software

The hardware components work together to form a complete anti-sway control system.Also, The camera detects the position of the ArUco marker, the computer processes this information using OpenCV, and the Arduino controls the motors based on the processed position data to stabilize the load and minimize oscillations.

In the next section, we will explore the software implementation that brings these hardware components to life, enabling the anti-sway functionality of the system.

2.7 Software Implementation

General description: Structurally, the system is divided into two parts that operate in parallel. The first subsystem is programmed using Python, and the second is programmed using the Arduino IDE in C. Communication is one-way and is carried out via a serial port. The general scheme of operation is shown in Figure 7.

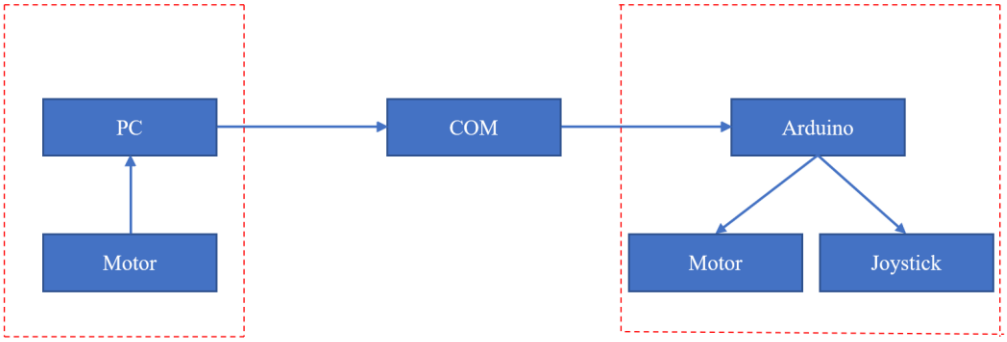

Figure 7: Schematic of the System

The presented model allows for the implementation of functions direct control of direct input devices with forbidden zones, object recognition, oscillation compensation, etc. The system is modular and allows you to add sensors and control devices to both the "right" and "left" parts.

Our setup allows for real-time position tracking and responsive actuation of the crane mechanism, combining computer vision, serial communication, and embedded motor control.

Description of the Python Code: The Python script was used to detect an ArUco marker using a webcam and to communicate its position over a serial connection to an Arduino. The base Block Diragram of the Python code you can find in Fig.8.

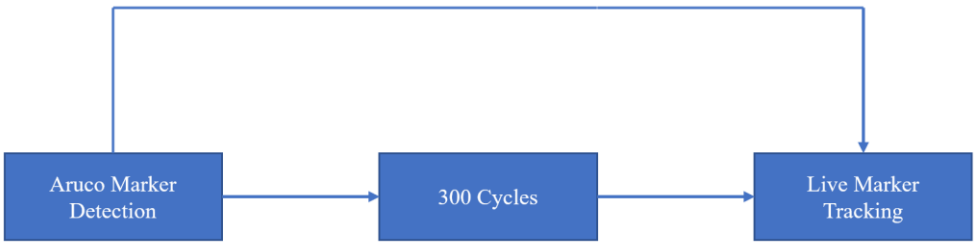

Figure 8: Block diagram of the Python Code

The code uses six libraries: cv2 and cv2.aruco, numpy, serial, time, and matplotlib.pyplot.

Camera Initialization The camera is set up to give the highest possible frame rate. Auto-exposure and autofocus are turned off to avoid constant changes in image settings and make detection more consistent. Gamma correction is applied using a custom function to improve contrast in bright lighting.

Marker Detection Function: A function called get_marker_center() is used to find the center coordinates (cx, cy) of the ArUco marker in a given frame. The marker is detected using a specific ID, and the image center is calculated by dividing the frame's width and height by two.

Calibration Phase: Before starting real-time tracking, the code collects 300 cx values to calculate an accurate average center position. This was necessary because the hardware was unstable and gave slightly different readings each time. Plotting these values helped visualize the calibration process.

Serial Communication: A connection is set up on COM4 at 115200 baud to communicate with the Arduino.

Main Loop: Inside the loop: A frame is captured from the camera. If the capture fails, a message is printed and the loop continues, if a frame is successfully captured, the script checks if the ArUco marker is visible, If the marker is found and calibration hasn't been completed yet, the script enters the calibration phase:

- It collects 300 cx values (x-coordinates of the marker center).
- The average of these 300 points is calculated to find a more stable center reference.
- This approach solves a previous problem where the marker center kept changing slightly due to hardware limitations. By averaging, the result becomes more stable and reliable.
- Once done, the calibration is marked complete, the calculated center is printed, and a plot of all 300 values is shown.

Post Calibration: A series of four values is sent to the Arduino through the serial port:

marker_flag, resolution, frame_center, current_cx

- marker_flag = 0 is used only once after calibration to send setup data.
- marker_flag = 1 is used afterward to indicate normal tracking mode (only cx is sent).

This flag helps the Arduino know how to interpret the received data.

Arduino Code Explanation: the Arduino code handles data received from the Python script and controls three motors based on joystick input and the position of an ArUco marker detected by the camera. It also includes an anti-sway mechanism that activates automatically after joystick movement stops (Fig.9).

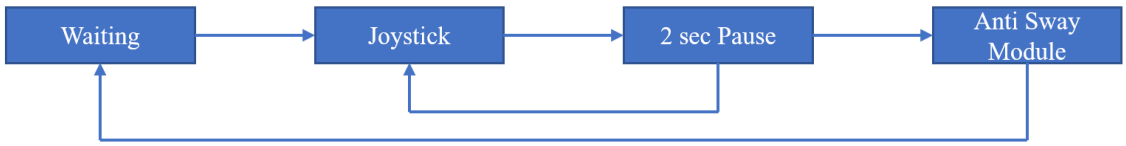

Figure 9: Block Diagram of Arduino Code

Initialization: The code first defines and initializes digital and PWM pins for controlling three motors, Analog pins are used to read values from a 3-axis joystick that controls the direction of the motors, A neutral value of 512 is defined for all joystick axes, based on the 10-bit ADC range (0–1023). A deadzone is applied around this value to filter out small unwanted signal fluctuations (noise), so motors only move when there's intentional input.

Main Loop (void loop ()): The program logic is handled using four main functions inside the main loop:

- ReadData (): Reads incoming serial data sent from the Python script (i.e., marker flag, frame resolution, center position, and current marker position cx).
- Joystick (): Interprets joystick movement and maps analog input values from 0–1023 to 0–255 PWM signals to control motor speed. It also detects if the joystick has been moved intentionally (i.e., input exceeds the deadzone).
- velocity_calculationX (int arucoPositionX):it is a function within the Arduino code implements a Proportional (P) control strategy. It calculates the speed of the trolley based on the horizontal displacement (error) between the center of the frame and the position of the ArUco marker. The control output (motor speed) is proportional to this error, scaled by a gain factor K_p. A larger gain of 5 is used for larger deviations to enable faster correction, while a smaller gain of 2.5 is applied for fine control near the center. This approach enhances response while preventing overshoot, resembling the behavior of a classical P-controller from the PID control family.

$$v = K_p * |x_{marker} - x_{center}|$$

- antisway (): Automatically moves the trolley in the opposite direction of the swing after joystick input has stopped. It’s designed to reduce the oscillation of the suspended load (crate) using a simple timed routine.

Control Logic Using Modes: The code uses a mode-based state machine to manage joystick input and anti-sway activation.

Mode 0: Idle / Joystick Monitoring: Continuously reads the joystick, if any axis moves outside the deadzone, it switches to Mode 1 (movementDetec = 1).

Mode 1: Active Manual Control: joystick controls the motors directly, once the joystick returns to the neutral position, it switches to Mode 2 (movementDetec = 0) and records the timestamp (t2 = millis()).

Mode 2: Stabilization Delay (Before Antisway): waits for 2 seconds (millis() - t2 > 2000) to allow natural sway to settle, then transitions to Mode 3.

*Mode 3: Antisway Mode:*Activates the antisway() function for 10 seconds to correct any remaining swing (millis() - t2 < 10000).

After 10 seconds, the system resets back to Mode 0 and is ready for new joystick input.

3. Results

In the meantime, we have a fully functioning system that consists of a joystick connected to the Arduino and a camera connected to the PC. The crane is fully controllable (crate, hook, and body), and an anti-sway algorithm was developed using ArUco marker detection to reduce system swinging. The measured oscillations with and without antisway module are presented in Fig 10.

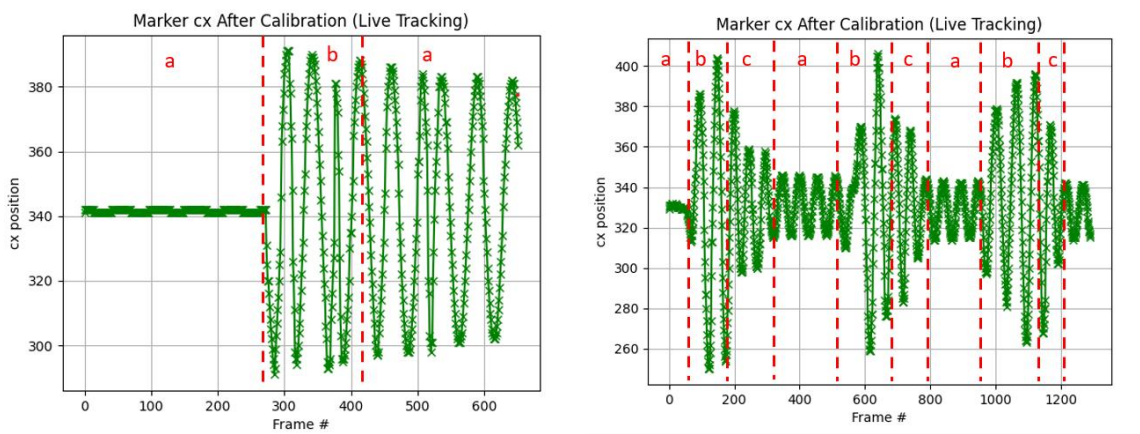

Figure 10: Result of measuring the position of the ArUco marker with and without antisway module.

a – waiting; b – joystick movement; c – antisway

4. Future Improvements

Second Camera with ArUco Markers for Z-Position Detection: The current system utilizes a single camera for horizontal position detection of the load. A significant improvement would be the implementation of a second camera mounted at the end of the crane's arrow.

This camera would be equipped to detect ArUco markers placed on the crate, enabling precise Z-coordinate position tracking.

Multi-Axis Control System Enhancements: The current anti-sway system focuses on single-axis control, primarily stabilizing the load during crate movement. To improve overall performance, the same anti-sway algorithm will be applied to the motor controlling the crane body. When the crane body moves (e.g., to the left), it causes the suspended load to swing. By integrating the anti-sway control into the body movement, this swing can be effectively minimized, resulting in more stable and precise operation across multiple axes.

5. Conclusion

The anti-sway control system for the scaled-down crane model has demonstrated effective performance in stabilizing loads and reducing oscillations. The integration of computer vision with motor control creates a responsive and adaptable system that significantly outperforms natural damping.

While challenges were encountered in camera calibration, marker detection, and motor control, these were largely addressed through careful system design and implementation. The remaining limitations, particularly related to motor response characteristics for small oscillations, have informed the direction for future improvements.

Numerous opportunities exist for expanding the system's capabilities through improved algorithms, enhanced vision systems, hardware upgrades, and multi-axis control.

The principles and techniques developed in this project have potential applications in education, industry, and research, demonstrating the broader relevance of anti-sway control systems beyond the specific implementation described here.

In summary, the anti-sway control system for the scaled-down crane model represents a successful integration of computer vision and control theory to solve a practical problem, with clear paths for future enhancement and application.

References

1. Ayman A. EI-Badawy, "Anti-sway control of a tower crane using inverse dynamics", IEEE Xplore: 22 January 2015

2. Wim Devesse; Marcus Ramteen; "A real-time optimal control method for swing-free tower crane motions" IEEE Xplore: 07 November 2013

3. T. Tocci, L. Capponi, and G. Rossi, "ArUco marker-based displacement measurement technique: Uncertainty analysis," *Engineering Research Express*, vol. 3, no. 3, p. 035032, Aug. 2021, doi: 10.1088/2631-8695/ac1fc7

4. Adrian Stein; David Vexler; Tarunraj Singh, "ArUco Based Reference Shaping for Real-time Precision Motion Control for Suspended Payloads", IEEE Xplore: 05 September 2024

5. Ivan Culjak; David Abram," A brief introduction to OpenCV" IEEE Xplore**:** 16 July 2012
6. Yusuf Abdullahi Badamasi," The working principle of an Arduino", IEEE *Xplore*: 29 December 2014
7. Sihang Feng; Yingqiang Liu, "Active Anti-Sway Control of Multi-Ropes Gantry Cranes with Scale Model Test", IEEE Xplore: 16 November 2023

Appendix A: Code Listings

A.1 Arduino Code

The following Arduino code implements the anti-sway control system for the crane. It handles joystick input for manual control and implements the anti-sway algorithm based on ArUco marker position data received from the Python script.

```
// Motor pins for the Trolley

// Motor pins
const int motor1Pin1 = 11;                    // Motor 1 Cranes Body
const int motor1Pin2 = 12;                    // For the Cranes Body
const int motor1PWMPin = 3;                   // Motor 1 PWM control pin for speed

const int motor2Pin1 = 8;                     // For the Hook
const int motor2Pin2 = 9;                     // For the Hook
const int motor2PWMPin = 6;                   // Motor 2 PWM control pin for speed

const int motor3Pin1 = 4;                     // For the Trolley (backward)
const int motor3Pin2 = 5;                     // For the Trolley (forward)
const int motor3PWMPin = 10;                  // Motor 3 PWM control pin for speed

// Joystick pins
const int joy1XPin = A2;                      // Joystick 1 X-axis (left-right)
const int joy2YPin = A1;                      // Joystick 2 Y-axis (up-down)
const int joy2XPin = A0;                      // Joystick 2 X-axis (forward-backward)

// Define dead zone threshold (for the Joystick)
const int deadZone = 80;

// Neutral joystick positions (these need to be adjusted or dynamically read)
int joy1XNeutral = 512;
int joy2YNeutral = 512;
int joy2XNeutral = 512;

const int tolerance = 20;                     // Tolerance range before movement
int arucoPositionX = -1;                      // Default to an invalid position (no
```

```
movement)

int velocity_calculation = 0;                    // Variable to store calculated velocity
int res=640;                                     //Camera resolution
int centerX=340;                                 //RAW centr position
int a,b,c,d;                                     // 4 int sent by Python through the serial port

void setup() {
   // Set motor pins as outputs
  pinMode(motor1Pin1, OUTPUT);
  pinMode(motor1Pin2, OUTPUT);
  pinMode(motor1PWMPin, OUTPUT);

  pinMode(motor2Pin1, OUTPUT);
  pinMode(motor2Pin2, OUTPUT);
  pinMode(motor2PWMPin, OUTPUT);
  // Set motor pins as outputs
  pinMode(motor3Pin1, OUTPUT);
  pinMode(motor3Pin2, OUTPUT);
  pinMode(motor3PWMPin, OUTPUT);

  // Initialize Serial for communication with the Python script
  Serial.begin(115200);
}

int mode = 0;
long t2 = 0;
long Timer = millis();
int movmentDetec = 0;

void loop() {
  ReadData ();
  long Timer = millis();

  if (mode !=3){
    int joy1X = analogRead(joy1XPin) - joy1XNeutral;
    int joy2Y = analogRead(joy2YPin) - joy2YNeutral;
    int joy2X = analogRead(joy2XPin) - joy2XNeutral;
    Joystick();
    if (abs(joy1X) > deadZone || abs(joy2Y) > deadZone || abs(joy2X) > deadZone){
      movmentDetec=1;
      }
    else{
      movmentDetec=0;
    }
    if (mode != 3 && movmentDetec==1){
```

```
      mode=1;
    }
    if(mode==1 && movmentDetec ==0){
      t2 = Timer;
      mode = 2;
    }
    if(mode == 2 && Timer-t2 > 2000){
      mode=3;
    }
  }
  if (mode == 3){
    antisway();
  }
}

void Joystick(){
  int joy1X = analogRead(joy1XPin) - joy1XNeutral;
  int joy2Y = analogRead(joy2YPin) - joy2YNeutral;
  int joy2X = analogRead(joy2XPin) - joy2XNeutral;


  // Set motor speed proportional to joystick input (map to a range 0-255)
  int speed1 = map(abs(joy1X), 0, 1023, 0, 255);
  int speed2 = map(abs(joy2Y), 0, 1023, 0, 255);
  int speed3 = map(abs(joy2X), 0, 1023, 0, 255);

  // Control motor 1 based on joystick 1 X-axis input (Cranes Body)

  if (abs(joy1X) > deadZone) {
    if (joy1X > 0) {
      // Move motor 1 Left
      digitalWrite(motor1Pin1, HIGH);
      digitalWrite(motor1Pin2, LOW);
    } else {
      // Move motor 1 Right
      digitalWrite(motor1Pin1, LOW);
      digitalWrite(motor1Pin2, HIGH);
    }
    analogWrite(motor1PWMPin, speed1);
  } else {
    // Stop motor 1
    digitalWrite(motor1Pin1, LOW);
    digitalWrite(motor1Pin2, LOW);
    analogWrite(motor1PWMPin, 0);
  }

  // Control motor 2 based on joystick 2 Y-axis input (Hook)
```

```
  if (abs(joy2Y) > deadZone) {
    if (joy2Y > 0) {
      // Move motor 2 Down
      digitalWrite(motor2Pin1, LOW);
      digitalWrite(motor2Pin2, HIGH);
    } else {
      // Move motor 2 Up
      digitalWrite(motor2Pin1, HIGH);
      digitalWrite(motor2Pin2, LOW);
    }
    analogWrite(motor2PWMPin, speed2);
  } else {
    // Stop motor 2
    digitalWrite(motor2Pin1, LOW);
    digitalWrite(motor2Pin2, LOW);
    analogWrite(motor2PWMPin, 0);
  }

  // Control motor 3 based on joystick 2 X-axis input (Trolley)
  if (abs(joy2X) > deadZone) {
    if (joy2X > 0) {
      // Move motor 3 Down
      digitalWrite(motor3Pin1, LOW);
      digitalWrite(motor3Pin2, HIGH);
    } else {
      // Move motor 3 Up
      digitalWrite(motor3Pin1, HIGH);
      digitalWrite(motor3Pin2, LOW);
    }
    analogWrite(motor3PWMPin, speed3);
  } else {
    // Stop motor 3
    digitalWrite(motor3Pin1, LOW);
    digitalWrite(motor3Pin2, LOW);
    analogWrite(motor3PWMPin, 0);
  }
}

void antisway(){
  //ReadData ();

  if (arucoPositionX != -1) {                    //prevent code from running on the first
circle
    velocity_calculationX(arucoPositionX);          //velocity calculation based on actual
position
```

```
 if (abs(arucoPositionX - centerX) >  tolerance) {         //determin the necessity of
mouvement
      if (arucoPositionX < centerX) {                        //determin the direction of
mouvement
        digitalWrite(motor3Pin1, LOW);                  // Stop backward motion
        digitalWrite(motor3Pin2, HIGH);                 // Move forward
        analogWrite(motor3PWMPin, velocity_calculation);  // Set speed based on velocity
calculation
      }
      if(arucoPositionX > centerX){                          //determin the direction of
mouvement
        digitalWrite(motor3Pin1, HIGH);                 // Move backward
        digitalWrite(motor3Pin2, LOW);                  // Stop forward motion
        analogWrite(motor3PWMPin, velocity_calculation);  // Set speed based on velocity
calculation
      }
    }
    if(abs(arucoPositionX - centerX) <=  tolerance){     //stop any mouvment if oscilation
smaller then tolerance
      digitalWrite(motor3Pin1, LOW);
      digitalWrite(motor3Pin2, LOW);
      analogWrite(motor3PWMPin, 0); // Stop the motor
    }
  }
}

// Function to calculate velocity
void velocity_calculationX(int arucoPositionX) {
  if (abs(arucoPositionX - centerX) >  tolerance) {                  // Corrected condition to
relate to centerX
    if(abs(arucoPositionX - centerX) > tolerance && abs(arucoPositionX -
centerX)<=tolerance+10){
    velocity_calculation = abs(arucoPositionX - centerX) * 5;         // Calculate velocity
based on distance from center
    }
    //if(abs(arucoPositionX - centerX)>tolerance+10 && abs(arucoPositionX -
centerX)<=50){

    if(abs(arucoPositionX - centerX)>tolerance+10 ){
    velocity_calculation = abs(arucoPositionX - centerX) * 2.5;         // Calculate velocity
based on distance from center
    }

  }
  else {
    velocity_calculation = 0;
  }
```

```
  if (velocity_calculation > 255) {                          // Ensure velocity does not exceed
PWM limits
    velocity_calculation = 255;
  }
}

void ReadData () {
  if (Serial.available() > 0) {                    // Check if data is available from the serial
port
    // it reads 4 int sent by Python through the serial port
    a = Serial.parseInt();                         // a is the marker 0 or 1
    b = Serial.parseInt();                         // Resolution
    c = Serial.parseInt();                         // centerX
    d = Serial.parseInt();                         // aruco position

    if (a == 0) {
      res = b;                                 // res is assigned to int b sent by Python (NOT IN
USE)
      centerX = c;                               // centerX is assigned to integer c
    }
    if (a == 1) {
      arucoPositionX = d;                          // Set aruco position to d
    }
  }
}
```

A.2 Python Computer Vision Code

The following Python code implements the ArUco marker detection and tracking system used in the anti-sway control system. It utilizes OpenCV for image processing and marker detection, with gamma correction to handle bright environments.

```
import cv2
import cv2.aruco as aruco
import numpy as np
import serial
import time
import matplotlib.pyplot as plt

# Function to adjust gamma for better contrast in bright environments
def adjust_gamma(image, gamma=0.1):
# Build a lookup table mapping the pixel values [0, 255] to their adjusted gamma values
    invGamma = 1.0 / gamma
    table = np.array([((i / 255.0) ** invGamma) * 255 for i in np.arange(0,
256)]).astype("uint8")
    return cv2.LUT(image, table)
```

```
# Function to detect marker center from given frame
def get_marker_center(frame, aruco_dict):
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Apply gamma correction to handle bright environments
    gray = adjust_gamma(gray, gamma=0.5)  # Reduce brightness

    corners, ids, rejected = aruco.detectMarkers(gray, aruco_dict)

    if ids is not None:
        for i in range(len(ids)):
            cx = int(np.mean(corners[i][0][:, 0]))
            cy = int(np.mean(corners[i][0][:, 1]))
            return cx, cy, corners, ids
    return None, None, None, None

# Initialize camera
cap = cv2.VideoCapture(0, cv2.CAP_DSHOW)  # Use DirectShow backend for speed
cap.set(cv2.CAP_PROP_FPS, 100)          # Request max FPS (100 FPS)
cap.set(cv2.CAP_PROP_AUTO_EXPOSURE, 0)   # Disable auto-exposure
cap.set(cv2.CAP_PROP_EXPOSURE, -9)       # Set very low manual exposure
cap.set(cv2.CAP_PROP_AUTOFOCUS, 0)       # Disable auto-focus
cap = cv2.VideoCapture(0)
if not cap.isOpened():
    raise Exception("Cannot open camera")

# Setup serial communication
ser = serial.Serial('COM4', 115200, timeout=1)
time.sleep(2)

# ArUco dictionary
aruco_dict = aruco.getPredefinedDictionary(aruco.DICT_4X4_50)

# Frame info
frame_width = int(cap.get(3))
frame_height = int(cap.get(4))
frame_center = (frame_width // 2, frame_height // 2)

# Variables
marker = 0
center = 0
k = 0
k2 = 0
calibration_limit = 300
```

```
cx_values_during_calibration = []
cx_values_after_calibration = []
# Start main loop
while True:
    ret, frame = cap.read()
    if not ret:
        print("Failed to grab frame")
        break

    cv2.circle(frame, frame_center, 5, (0, 0, 255), -1)

    #  Get current marker center using the SAME frame
    cx, cy, corners, ids = get_marker_center(frame, aruco_dict)

    if ids is not None and cx is not None:
        aruco.drawDetectedMarkers(frame, corners, ids)

        # Calibration phase
        if k == 0:
            print("Calibrating center...")
            while k2 < calibration_limit:
                ret, frame = cap.read()
                if not ret:
                    continue

                cx, cy, corners, ids = get_marker_center(frame, aruco_dict)
                if cx is not None:
                    cx_values_during_calibration.append(cx)
                    print(f"Frame {k2+1}: cx = {cx}")

                k2 += 1
                time.sleep(0.03)

            if len(cx_values_during_calibration) > 0:
                center = int(np.mean(cx_values_during_calibration))
                print(f" Calibration complete.")
                print(f" Calculated center (mean of {calibration_limit} values): {center}")
            else:
                center = frame_width // 2  # fallback

            #  Plot calibration cx values
            plt.figure("Calibration Phase")
            plt.plot(cx_values_during_calibration, marker='o', linestyle='-', color='blue')
            plt.axhline(y=center, color='r', linestyle='--', label=f"Center = {center}")
            plt.title("Marker cx during Calibration (Mean-Based)")
            plt.xlabel("Frame #")
```

```
            plt.ylabel("cx position")
            plt.legend()
            plt.grid()
            plt.show()

        k = 1  # calibration complete

        # After calibration
        cv2.circle(frame, (cx, cy), 5, (255, 0, 0), -1)
        message = f"{marker} {frame_width} {center} {cx}\n"
        ser.write(message.encode('utf-8'))
        print("[]", message.strip())

        cx_values_after_calibration.append(cx)

        marker = 1
        frame_width = 0
        center = 0

    cv2.imshow('ArUco Marker Detection', frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

# Final analysis
cap.release()
cv2.destroyAllWindows()
ser.close()

#  Plot post-calibration cx tracking
if cx_values_after_calibration:
    plt.figure("Post-Calibration Movement")
    plt.plot(cx_values_after_calibration, marker='x', linestyle='-', color='green')
    plt.title("Marker cx After Calibration (Live Tracking)")
    plt.xlabel("Frame #")
    plt.ylabel("cx position")
    plt.grid()
    plt.show()
```

Appendix B: Component Specifications

B.1 Arduino Uno

- Microcontroller: ATmega328P
- Operating Voltage: 5V
- Input Voltage (recommended): 7-12V
- Digital I/O Pins: 14 (of which 6 provide PWM output)

- Analog Input Pins: 6
- DC Current per I/O Pin: 20 mA
- DC Current for 3.3V Pin: 50 mA
- Flash Memory: 32 KB (ATmega328P) of which 0.5 KB used by bootloader
- SRAM: 2 KB (ATmega328P)
- EEPROM: 1 KB (ATmega328P)
- Clock Speed: 16 MHz
- Length: 68.6 mm
- Width: 53.4 mm
- Weight: 25 g

B.2 H-Bridge Motor Driver

- Model: L298N Dual H-Bridge Motor Driver
- Motor Supply Voltage (Vs): 5V to 35V
- Logic Supply Voltage (Vss): 5V
- Peak Output Current: 2A per channel
- Continuous Output Current: 1.5A per channel
- Logic Input Voltage: High: 2.3V (min), Low: 1.5V (max)
- Maximum Power Dissipation: 25W
- Operating Temperature: -25°C to +130°C

B.3 ELP USB Camera

- Model: ELP-USBFHD01M-L180
- Sensor: 1/3" CMOS OV2710
- Resolution: 1920 x 1080 (Full HD)
- Frame Rate: Up to 100fps @720P, 60fps @1080P
- Lens: 180-degree fisheye lens
- Interface: USB 2.0
- Power Supply: DC 5V (USB powered)
- Dimensions: 38mm x 38mm x 22mm
- Operating System Compatibility: Windows, Linux, Android, Mac OS

B.4 DC Motors

- Operating Voltage: 9V DC
- No-load Speed: Approximately 200 RPM
- Stall Torque: Approximately 0.5 kg-cm
- Gear Ratio: 1:48
- Current Draw: 70mA (no load), up to 250mA (stall)